WUNDERDOG

Admicom x Wunderdog – Evals & Cloud Agents

DevDays 20.–21.10.2026 @ Jyväskylä

Workshop facilitators



Juuso Korhonen

Technology Lead

- 10+ years of full stack
- 15+ years of programming
- Technical Lead specializing in rapid idea-to-production delivery within weeks
- React, React Native, Typescript, Node, Python, PostgreSQL, Contentful, GCP, AWS, Terraform



Mikko Vallaskivi

Technology Lead

- Software development generalist
- 10+ years of full stack web, mobile and cloud experience
- 15+ years of programming
- Copilot/Codex/Claude Code

Two themes

01

Evaluations

Know when it works

Trust → autonomy

02

Cloud agents

Let it work while you don't

WUNDERDOG

01

Evaluations

Vibes don't scale

PROMPT Catalogue last week's review comments by problem type



Same prompt. Different outcome.

Anatomy of an eval



Task

Input and success criteria



Trials

Several attempts per task

Transcript

Every step the agent took

Outcome

Final state of the world

tests pass

skill used

rubric

Graders

Each checks one thing

67%

Score

Pass rate across trials

Grade the outcome, not the claim

WHAT THE AGENT SAYS

Fixed the flaky test. CI is green.

CLAIM ≠ OUTCOME

WHAT THE REPO SAYS

```
$ run-tests auth.spec --repeat 10
✓ 7 runs passed
× 3 runs failed   token expiry
```

- Check the end state
- Not the agent's report

Three kinds of graders

Code

Unit tests · regex · tool-call checks

+ Fast, cheap, repeatable
– Brittle on nuance

Model

Rubrics · pairwise · judge panels

+ Handles open-ended output
– Needs calibration

Human

Expert review · spot checks

+ Gold standard
– Slow and expensive

Prefer deterministic

Escalate only when needed

pass@k vs pass^k

Run the same task k times. Does any run succeed, or does every run?

pass@k

$1 - (1 - p)^k$

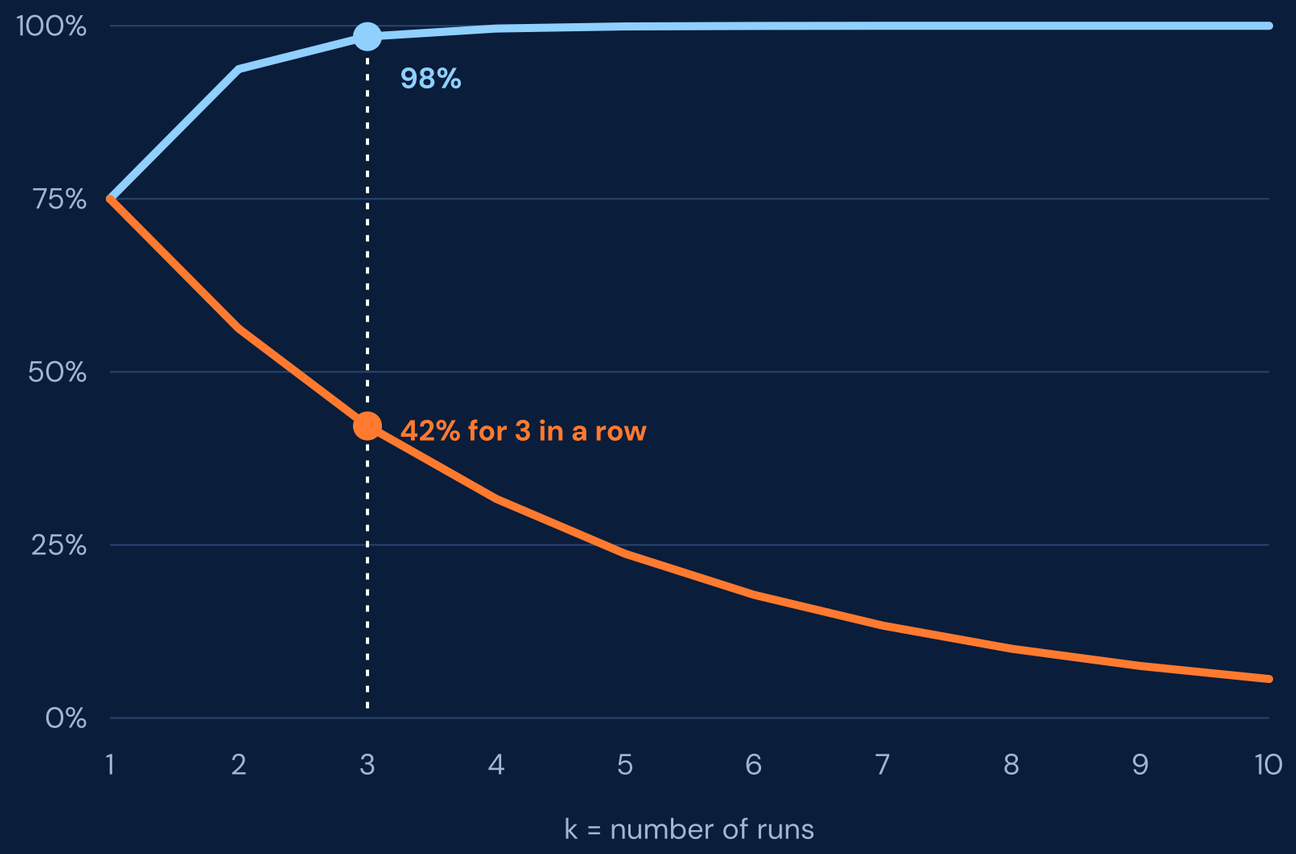
At least one run succeeds

pass^k

p^k

Every run succeeds

p 75%



Capability vs regression

Capability

What can it do yet?



64%

Regression

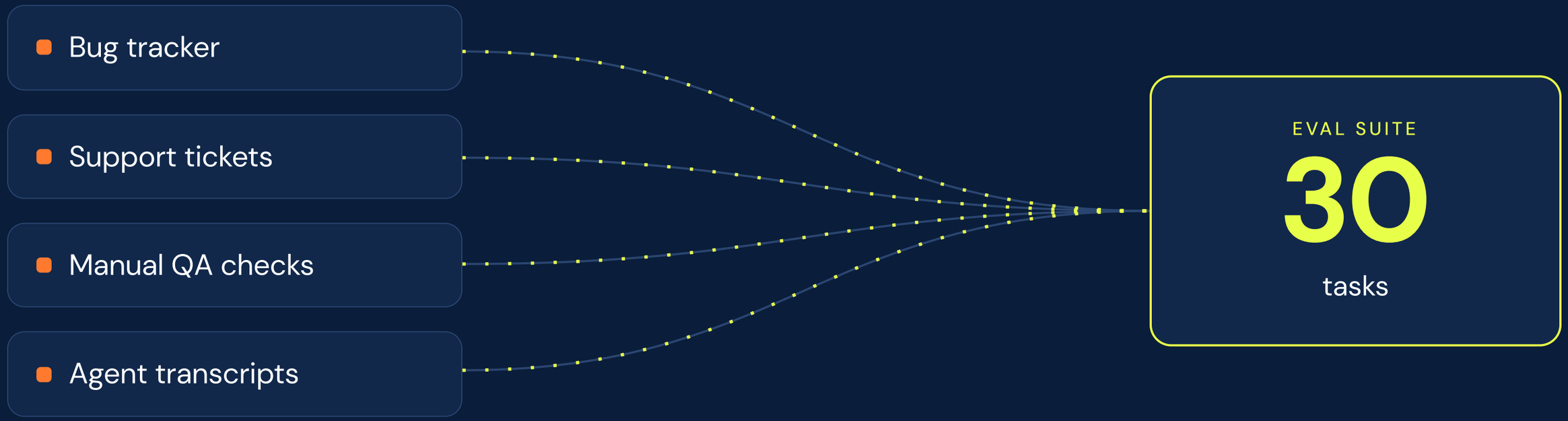
Does it still work?

`rule-not-followed` GRADUATES WHEN IT PASSES RELIABLY



98%

Start from real failures



20–50 tasks is enough to start

Anatomy of a plugin eval

```
evals/
└─ rule-not-followed/
   ├─ prompt.md
   └─ graders/
      ├─ finding.md
      └─ skill-fired.md
```

prompt.md

```
---
max_turns: 20
allowed_tools: [Read, Skill]
---
Catalogue the review comments in comments.json.
Write your findings to findings.json.
```

finding.md

```
type: regex
target:
  source: file
  path: findings.json
pattern:
  'rule-not-followed'
```

skill-fired.md

```
type: tool_used
tool: Skill
input_match:
  'review-catalogue'
```

● Grades the result ● Grades the path

Six grader types

FREE · FROM TRANSCRIPT AND FILES

`regex`	Pattern in reply, trace or file
`tool_used`	Tool called, n times
`tool_order`	A before B
`file_exists`	File was created

JUDGE MODEL · COSTS A CALL

`llm`	Rubric, 2 of 3 votes
`baseline`	At least as good as a reference transcript

One grader on the result.
One on the path.

Run it

```
$ claude plugin eval init
drafts cases and graders, tries them once

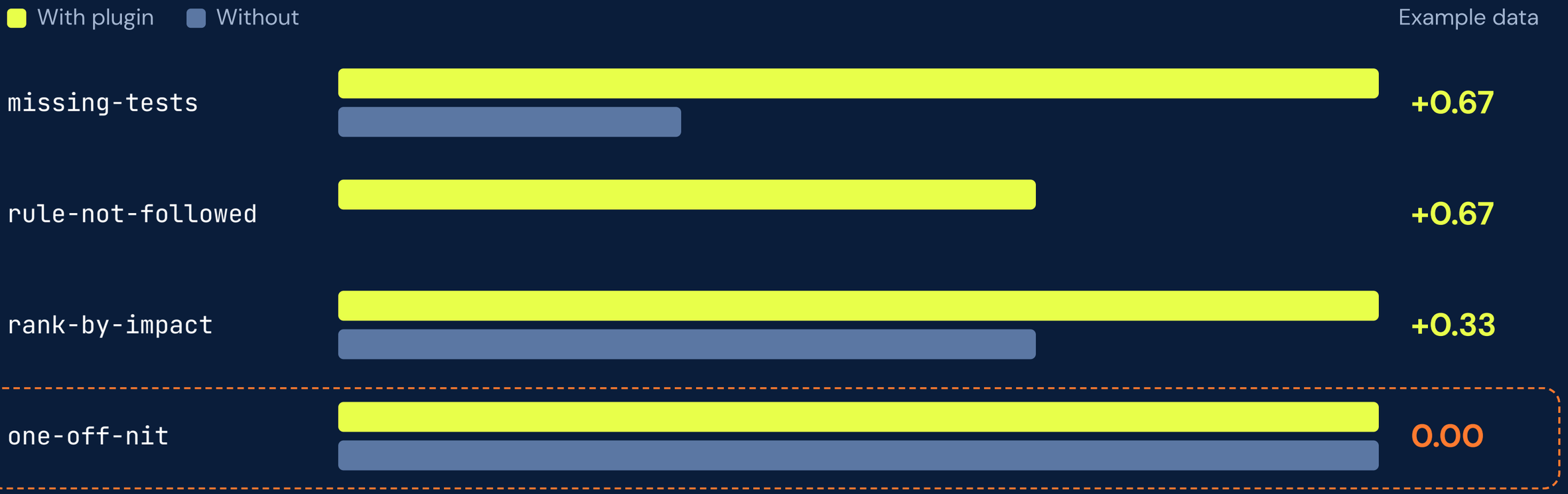
$ claude plugin eval . --allow-tools Write

CASE                    WITH    W/OUT   Δ
missing-tests           1.00    0.33    +0.67
rule-not-followed       0.67    0.00    +0.67
rank-by-impact          1.00    0.67    +0.33
one-off-nit             1.00    1.00    0.00

report → evals/results/<timestamp>/report.html
```

- 3 runs per case
- Baseline runs without the plugin
- Gate CI on the threshold

Read the delta



Passes without the plugin → the plugin isn't why it passes

Test both directions

SHOULD TRIGGER

- What do reviewers keep flagging in our PRs?
- Group last week's review comments by theme
- Which review comments should become rules?

SHOULD NOT TRIGGER · NEAR MISSES

- Review this pull request
- Reply to the comments on PR #412
- Write release notes from last week's PRs

~20 queries | 60 / 40 train · test | 3 runs per query | Pick the description by held-out score

Pitfalls

Ambiguous task

Two experts must agree on pass or fail

Rigid grader

`"96.12"` rejected for `96.124991…`

42% → **95%** after fixing grading and task bugs

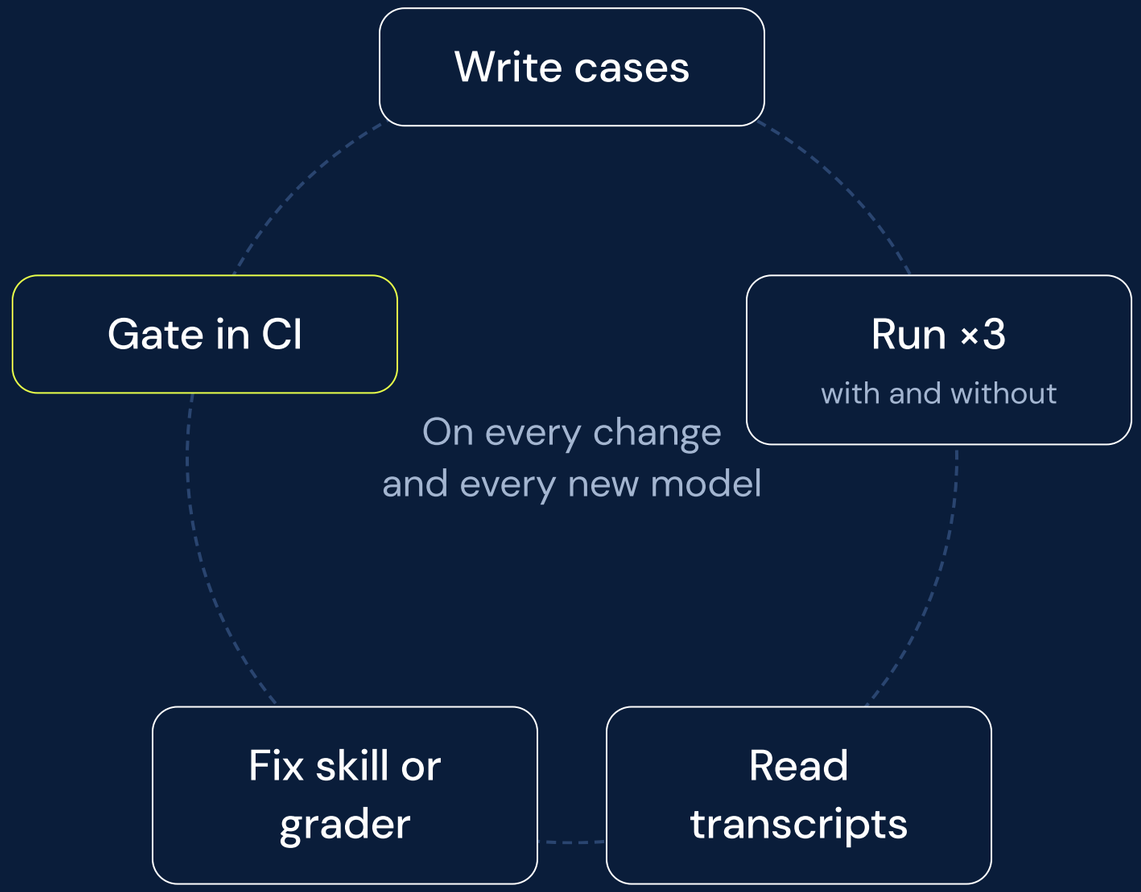
Shared state

Leftover files inflate scores between runs

Unread transcripts

Read them. Every failure, every time.

The eval loop



Define done first

An eval is a spec you can run.

- Start small, from real failures
- Grade outcomes, read transcripts
- Measure what the skill adds

WUNDERDOG

02

Cloud agents

From pair programmer to teammate



Autocomplete	Chat	Local agent	Cloud agent	Routines
Suggests the next line	Answers questions	Edits and runs code	Works while you're away	Starts on its own triggers
You type	You ask	You watch	You review	You set the goal

Where agents run

Local session

Your machine, your files, your tools

`claude`

Remote Control

Your machine, steered from phone or browser

`claude remote-control`

Self-hosted

Your infrastructure, cloud workflow

`self-hosted environment`

Cloud session

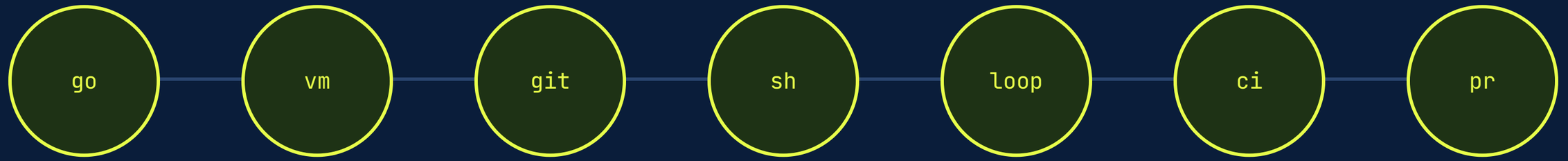
Managed VM, keeps going when the laptop closes

`claude --cloud`

Runs on your hardware

Runs in managed cloud

Anatomy of a cloud session



Trigger
Person, schedule, API, PR

Fresh VM
Isolated per session

Clone
Repo at branch

Setup
Script, cached

Agent loop
Edit, run, test

Checks
Tests and lint

Pull request
On a claude/ branch

SAME SHAPE — Claude Code · Codex Cloud · Copilot cloud agent · Cursor Cloud Agents

The environment is the contract

NETWORK ACCESS

None | **Trusted** | Custom | Full

SETUP SCRIPT CACHED

```
npm ci && npx playwright install
```

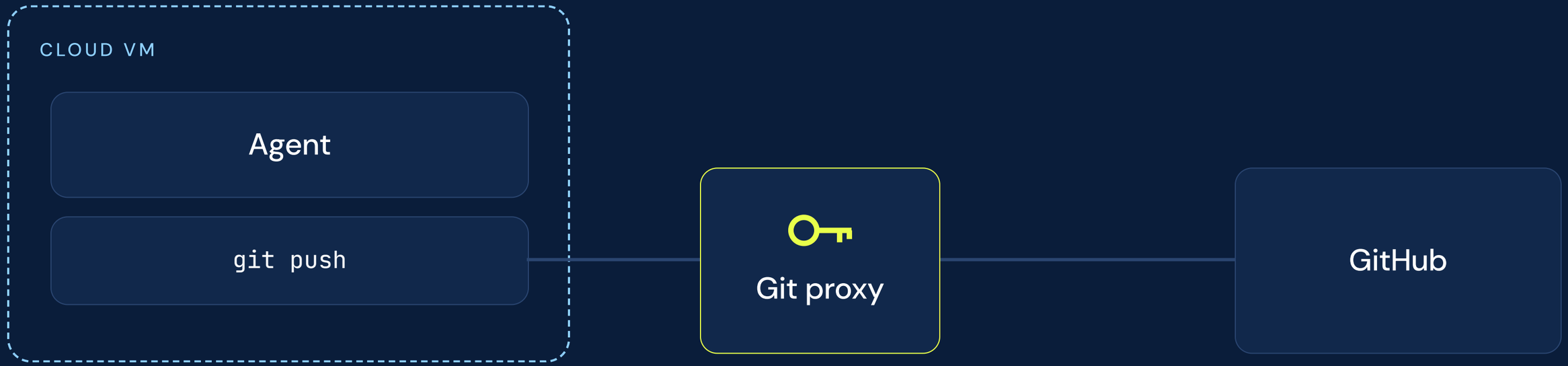
VARIABLES

```
NODE_ENV=test
```

registry.npmjs.org	allowed
pypi.org	allowed
api.yourcompany.com	blocked
random-paste-site.io	blocked

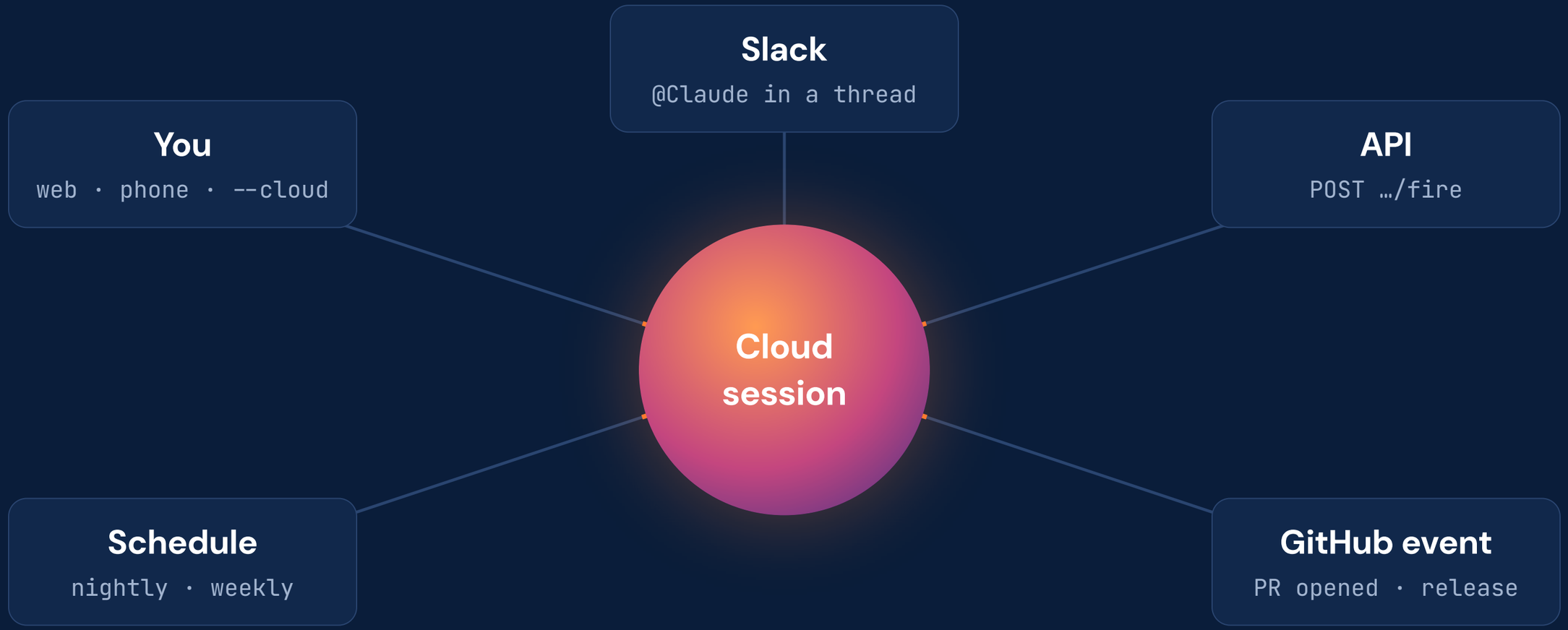
Open only what the task needs

Credentials stay outside



- Tokens never enter the VM
- No tag pushes or branch deletes
- Branch protection rules still apply

What starts a cloud agent



Fan out

```
$ claude --cloud "Triage failed run 8812 on main"
$ claude --cloud "Catalogue last week's reviews"
$ claude --cloud "Upgrade date-fns to v4"
```



- One VM per task
- Parallel runs share your usage

Plan locally, run in the cloud

LOCAL

Plan

```
claude --permission-
mode plan
```

→

LOCAL

Commit the plan

```
docs/migration-
plan.md
```

→

CLOUD

Execute

```
claude --cloud
"Execute
docs/migration-
plan.md"
```

→

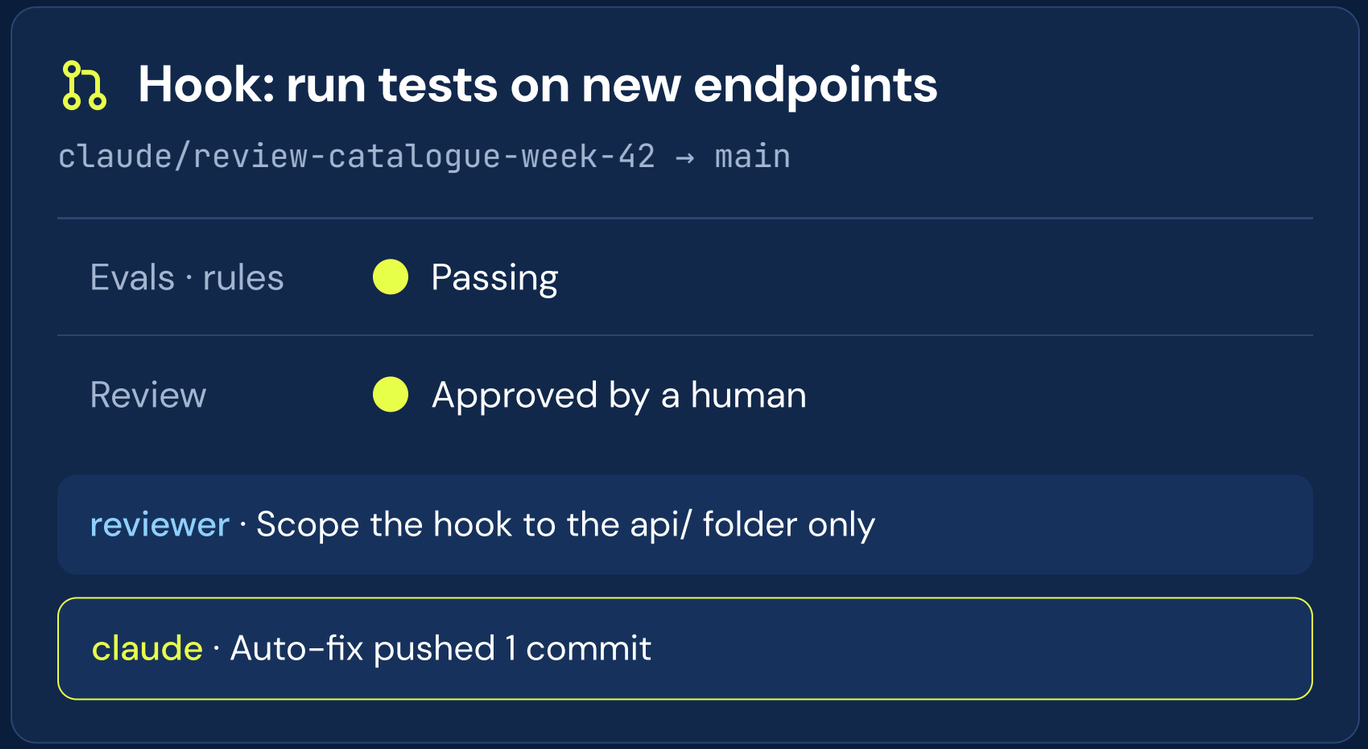
LOCAL

Pull it back

```
claude --teleport
```

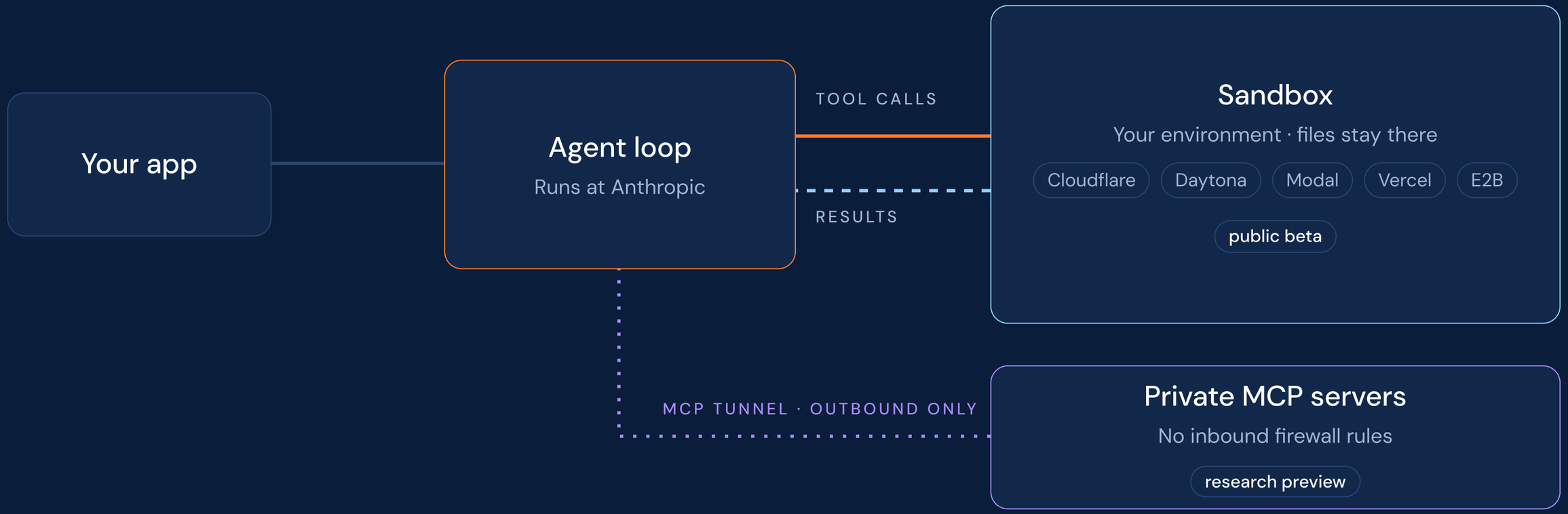
- Talk where feedback is fast
- Run where waiting is free

Contributors, not committers



- Agent branch, human merge
- Auto-fix answers CI and review comments
- Branch protection stays on

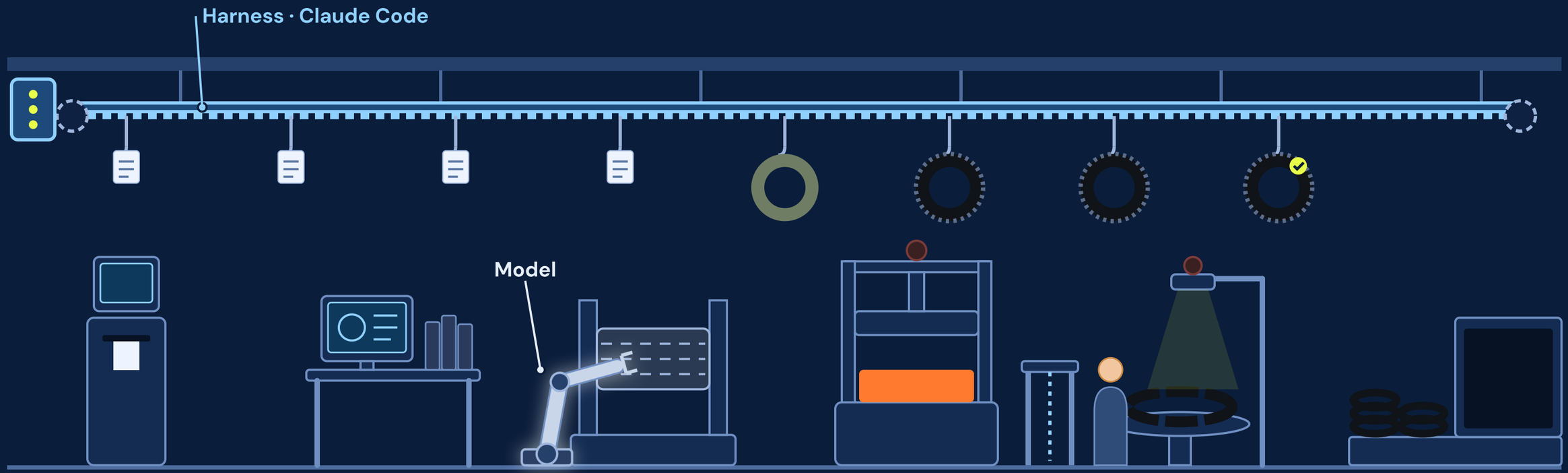
Build your own with Managed Agents



03

One plant, two lines

Meet the tire plant



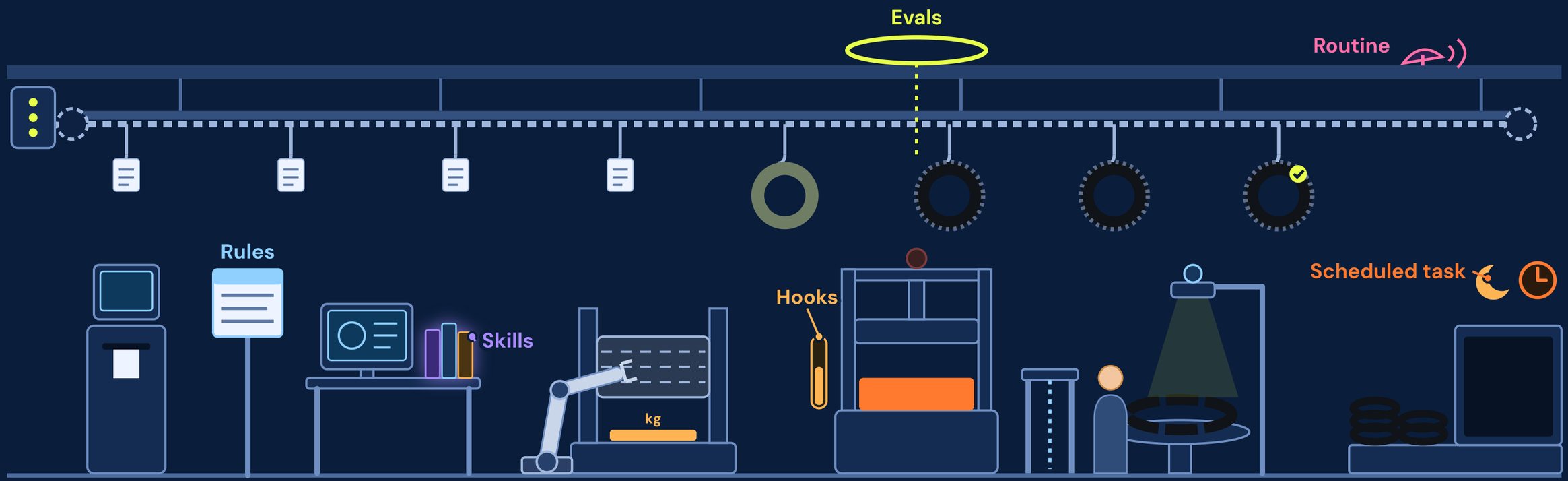
TIRE PLANT

Robot arm does the building; swap it and the line still runs

AGENTIC SDLC

Model does the work; the harness stays the same

What runs the plant



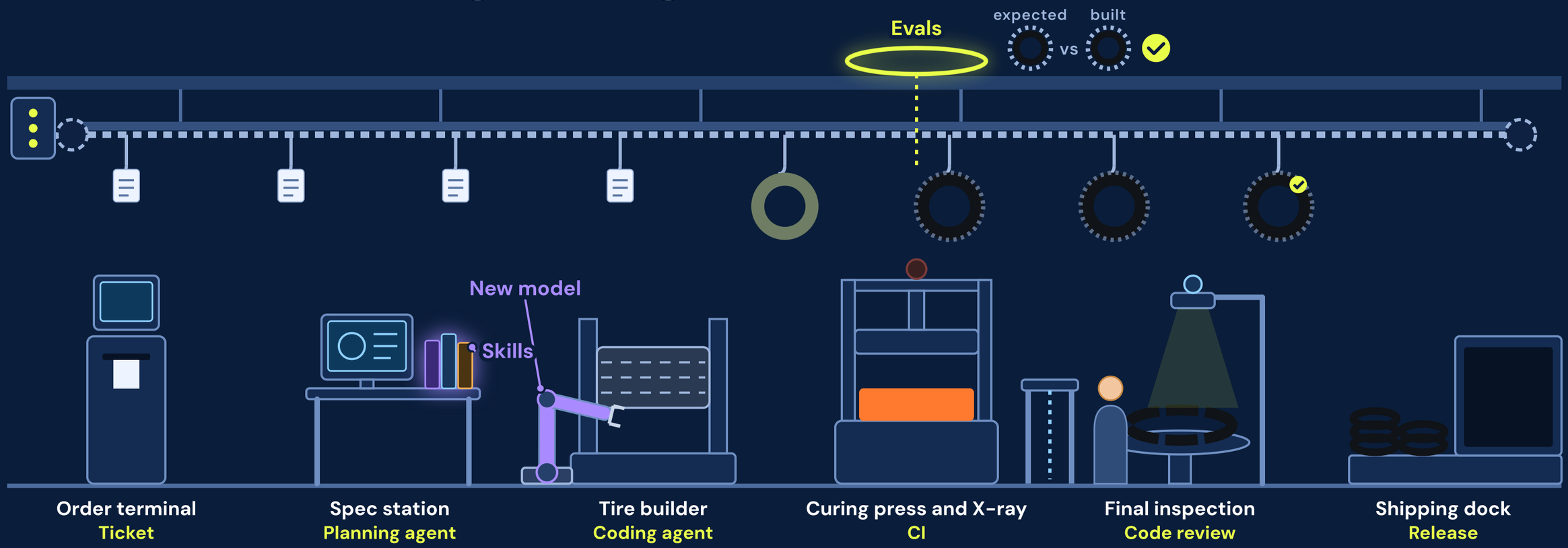
TIRE PLANT

Control centre runs the plant while you are off site

AGENTIC SDLC

Routine in the cloud, on a schedule, an API call or a GitHub event

When something changes



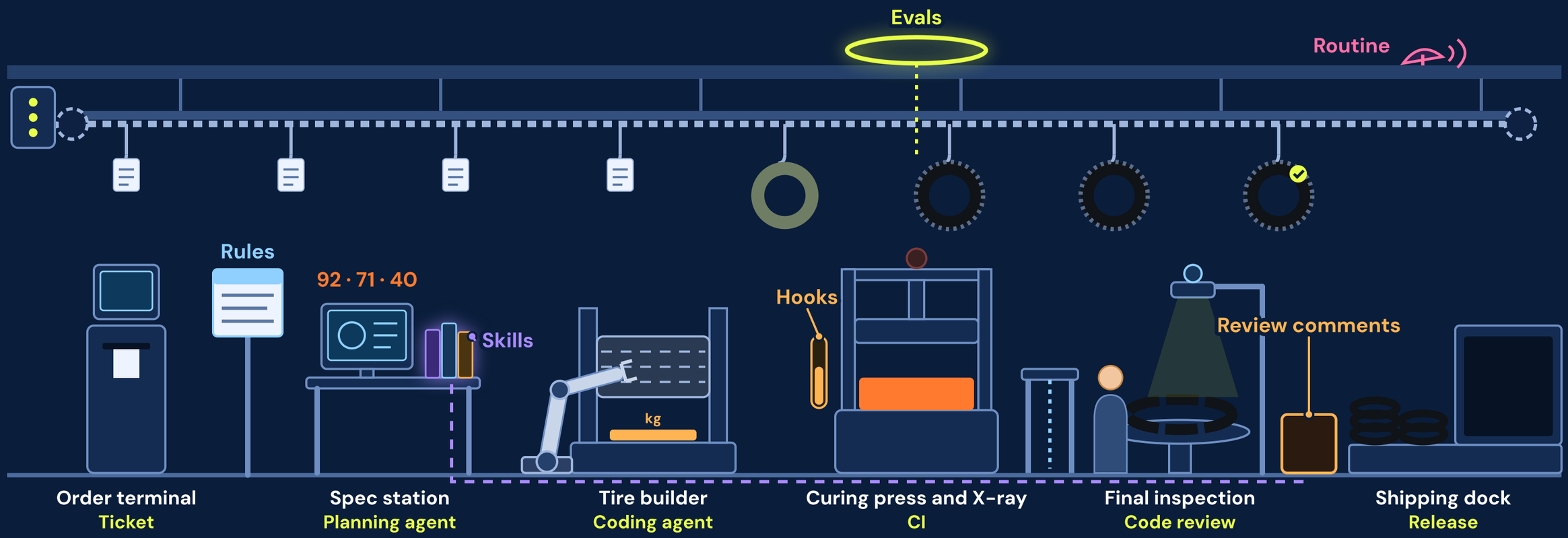
TIRE PLANT

Recipe updated for the new robot: right tread at the new speed

AGENTIC SDLC

Skill updated, evals green keep the speed-up, lose the drift

Line 1: Review-comment cataloguer



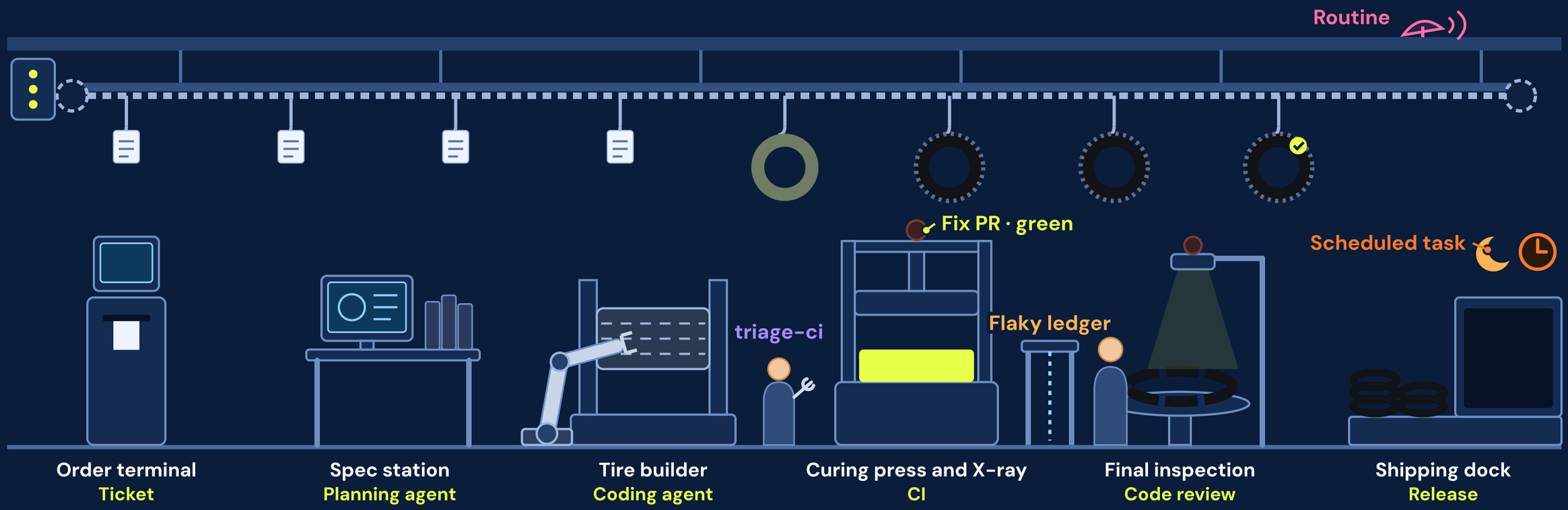
TIRE PLANT

Weekly from the control centre nobody has to remember

AGENTIC SDLC

Routine weekly: a retro board summary and a draft PR

Line 2: CI health



TIRE PLANT

Repair order press back in spec

AGENTIC SDLC

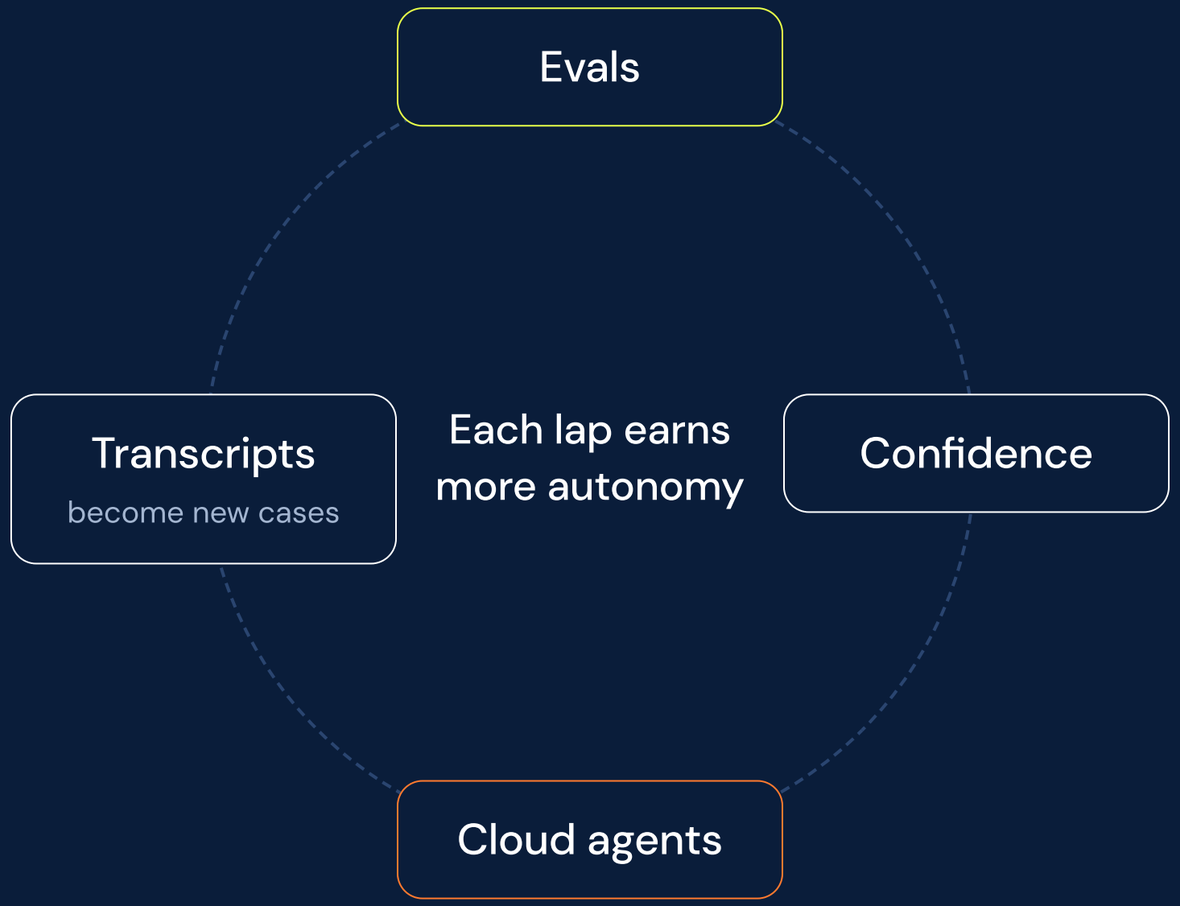
Draft fix PR a human reviews it; main is green again

From skill to night shift to routine

	Skill On demand	Scheduled task Your machine	Routine Cloud
Review catalogue	`/catalogue-reviews` Before a retro	Friday afternoon: last week's comments	Weekly schedule: retro board and a draft PR
CI health	`/triage-ci <run>` When a build fails	Every morning: scan main, update the ledger	CI failure calls the API trigger; nightly ledger

Evals gate every step up

Trust before autonomy



More lines for the same plant

Library updates

One upgrade per PR, changelog read, tests green

`schedule · weekly`

SDLC observability

Alert in, investigation done, draft fix PR out

`API trigger · on alert`

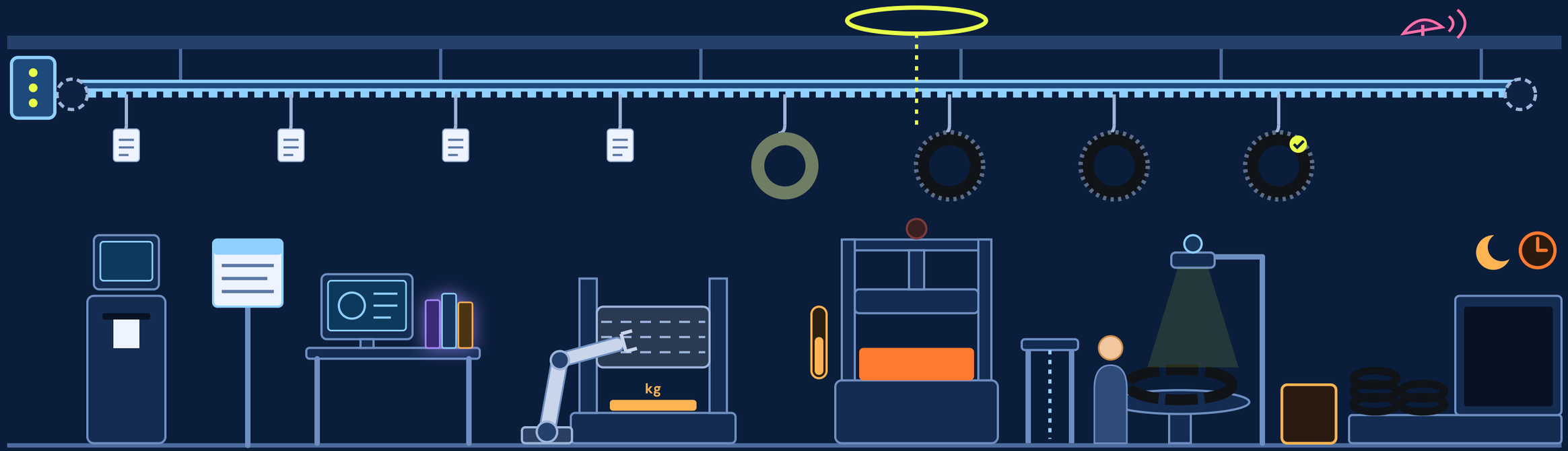
Migrations at scale

Playbook as a skill, evals first, then fan out

`fan-out · per module`

Same machines. Same control system. New recipe.

Your shift starts now



TIRE PLANT

Then your own plant the same machines on your own line

AGENTIC SDLC

Then your own team apply it to a real problem from your own context

Thank you

WUN
DER
DOG